

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 2 – SCENARIO BASED

|                  |                                                                                                                            |               |                                    |
|------------------|----------------------------------------------------------------------------------------------------------------------------|---------------|------------------------------------|
| Course Code:     | DSA03                                                                                                                      | Course Title: | Natural Language Processing        |
| CO Assessed:     | CO2 – Manipulation                                                                                                         | Assessment:   | Assessment Tool 2 – SCENARIO BASED |
| Weightage:       | 35% of CIA                                                                                                                 |               |                                    |
| CO5 Statement:   | Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3) |               |                                    |
| Student Name:    | Mohamed Jameel A                                                                                                           | Reg. No.:     | 192425057                          |
| Section / Batch: | 2024                                                                                                                       | Date:         |                                    |

### Code of Conduct

I Mohamed Jameel A/192425057 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

### Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

| Section / Topic                                                                   | Focus                                                                                                                                              | Q Nos |
|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM | Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation. | Q1    |

## Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any builtin NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**

- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

### Training Corpus

| Sentence                                 | POS Tags      |
|------------------------------------------|---------------|
| The/DT boy/NN eats/VBZ rice/NN           | DT NN VBZ NN  |
| The/DT girl/NN drinks/VBZ milk/NN        | DT NN VBZ NN  |
| A/DT cat/NN drinks/VBZ milk/NN           | DT NN VBZ NN  |
| The/DT dog/NN chases/VBZ cat/NN          | DT NN VBZ NN  |
| A/DT teacher/NN teaches/VBZ students/NNS | DT NN VBZ NNS |
| Students/NNS study/VBP English/NN        | NNS VBP NN    |
| Birds/NNS fly/VBP high/RB                | NNS VBP RB    |
| Children/NNS play/VBP games/NNS          | NNS VBP NNS   |

### New Input Sentence

The cat drinks milk

### Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

### Rubrics for Calculation

| Criteria                     | Weightage (%) | Level 4 (Exemplary)                                                       | Level 3 (Proficient)                               | Level 2 (Developing)                              | Level 1 (Beginning)                               |
|------------------------------|---------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%           | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%           | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%           | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%           | Makes wellreasoned, logical decisions with strong rationale               | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%           | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |

| Q. No. /<br>Criteria<br>Level | Understanding<br>of Scenario | Identification<br>of Key Issues | Application<br>of<br>Concepts | Decision<br>Making | Clarity of<br>Response | Sub.<br>Total | Total<br>% |
|-------------------------------|------------------------------|---------------------------------|-------------------------------|--------------------|------------------------|---------------|------------|
| 1                             |                              |                                 |                               |                    |                        |               |            |
| 2                             |                              |                                 |                               |                    |                        |               |            |
| 3                             |                              |                                 |                               |                    |                        |               |            |
| 4                             |                              |                                 |                               |                    |                        |               |            |
| 5                             |                              |                                 |                               |                    |                        |               |            |

| Faculty In-charge               | Course Coordinator              | Program Director                |
|---------------------------------|---------------------------------|---------------------------------|
| Signature: _____<br>Date: _____ | Signature: _____<br>Date: _____ | Signature: _____<br>Date: _____ |

## Q1. Morphological Parsing

| Word      | Prefix | Base Form | Suffix | Morphological Analysis | Semantic Effect                                                                |
|-----------|--------|-----------|--------|------------------------|--------------------------------------------------------------------------------|
| disagree  | dis-   | agree     | —      | dis + agree            | Reverses/negates the meaning of <i>agree</i>                                   |
| agreement | —      | agree     | -ment  | agree + ment           | Changes the verb <i>agree</i> into a noun meaning the state/result of agreeing |
| agreeable | —      | agree     | -able  | agree + able           | Changes the meaning to something/someone that is acceptable or pleasing        |

### 1. disagree

#### Structure:

dis- + agree

- **Prefix:** dis-
- **Base:** agree
- **Suffix:** None
- **Semantic effect:** dis- gives a negative or opposite meaning.

agree → disagree

---

### 2. agreement

#### Structure:

agree + -ment

- **Prefix:** None
- **Base:** agree
- **Suffix:** -ment
- **Semantic effect:** -ment forms a noun referring to the **state or result of agreeing**.

agree → agreement

---

### 3. agreeable

#### Structure:

agree + -able

- **Prefix:** None
- **Base:** agree
- **Suffix:** -able
- **Semantic effect:** -able forms an adjective expressing **a quality or suitability**.

agree → agreeable

## Python Program

# Morphological Parsing

words = {

```

"disagree":
    { "prefix": "dis-",
      "base": "agree",
      "suffix": "-"},
    },
"agreement":
    { "prefix": "-",
      "base": "agree",
      "suffix": "-ment"},
    },
"agreeable":
    { "prefix": "-",
      "base": "agree",
      "suffix": "-able"},
    }
}

```

```

for word, parts in words.items():
    print("Word:", word)    print("Prefix:",
    parts["prefix"])    print("Base:",
    parts["base"])    print("Suffix:",
    parts["suffix"])    print()

```

### Output

```

Word: disagree
Prefix: dis- Base:
agree
Suffix: -

```

```

Word: agreement
Prefix: - Base:
agree
Suffix: -ment

```

```

Word: agreeable
Prefix: - Base:
agree Suffix:
-able

```

### Conclusion

The three words demonstrate **derivational morphology** because prefixes and suffixes are added to the base word **agree**, changing its meaning and/or grammatical category.

## Q2. Emission Probability

The corpus gives us words with their POS tags, for example:

The/DT boy/NN eats/VBZ rice/NN

...

The **emission probability** tells us:

How likely is a particular word to be generated by a particular POS tag? **Formula**

The corpus contains 8 sentences. **For**

**the words in the new sentence** New

sentence:

The cat drinks milk

### 1. The

The appears twice, both times as DT.

DT occurs 5 times.

**2. cat** cat appears twice, both times as NN.

NN occurs 8 times.

**3. drinks** drinks appears twice,  
both times as VBZ.

VBZ occurs 5 times.

**4. milk** milk appears twice, both  
times as NN.

### Final table

#### Word POS Count(word, POS) Count(POS) Emission Probability

|        |     |   |   |             |
|--------|-----|---|---|-------------|
| The    | DT  | 2 | 5 | <b>0.40</b> |
| cat    | NN  | 2 | 8 | <b>0.25</b> |
| drinks | VBZ | 2 | 5 | <b>0.40</b> |
| milk   | NN  | 2 | 8 | <b>0.25</b> |

---

## Python Program

# Emission Probability

```
corpus = [  
    [("The", "DT"), ("boy", "NN"), ("eats", "VBZ"), ("rice", "NN")],  
    [("The", "DT"), ("girl", "NN"), ("drinks", "VBZ"), ("milk", "NN")],  
    [("A", "DT"), ("cat", "NN"), ("drinks", "VBZ"), ("milk", "NN")],  
    [("The", "DT"), ("dog", "NN"), ("chases", "VBZ"), ("cat", "NN")],  
    [("A", "DT"), ("teacher", "NN"), ("teaches", "VBZ"), ("students", "NNS")],  
    [("Students", "NNS"), ("study", "VBP"), ("English", "NN")],  
    [("Birds", "NNS"), ("fly", "VBP"), ("high", "RB")],  
    [("Children", "NNS"), ("play", "VBP"), ("games", "NNS")]
```

]

```
word_tag_count = {} tag_count  
= {}
```

```
for sentence in corpus:
```

```
for word, tag in sentence:
```

```
    word_tag_count[(word, tag)] = word_tag_count.get((word, tag), 0) + 1
```

```
tag_count[tag] = tag_count.get(tag, 0) + 1
```

```
test_words = ["The", "cat", "drinks", "milk"]
```

```
for word in test_words:
```

```
for tag in tag_count:
```

```
    count = word_tag_count.get((word, tag), 0)
```

```
    if count > 0:
```

```
        probability = count / tag_count[tag]
```

```
print(word, tag, probability)
```

### Q3 – Transition Probability

The **transition probability** tells us how likely one POS tag is to be followed by another POS tag. **Formula**

From the given corpus, the common transitions are:

DT → NN

NN → VBZ

NN → NNS

NN → END

VBZ → NN

NNS → VBP

NNS → END

VBP → NN

VBP → RB

VBP → NNS

RB → END

The corpus is given in the assessment itself.

**Example: DT → NN**

Every DT in the corpus is followed by NN.

There are **5 DT tags**, and all 5 are followed by NN.

**Example: NN → VBZ** There

are 8 NN tags.

Out of these, 5 are followed by VBZ.

**Example: NNS → VBP** There

are 5 NNS tags.

3 are followed by VBP.

### Important transition probabilities

#### Previous → Next Probability

|           |                    |
|-----------|--------------------|
| DT → NN   | 5/5 = <b>1.00</b>  |
| NN → VBZ  | 5/8 = <b>0.625</b> |
| NN → NNS  | 1/8 = <b>0.125</b> |
| VBZ → NN  | 4/5 = <b>0.80</b>  |
| NNS → VBP | 3/5 = <b>0.60</b>  |
| VBP → NN  | 2/4 = <b>0.50</b>  |
| VBP → NNS | 1/4 = <b>0.25</b>  |
| VBP → RB  | 1/4 = <b>0.25</b>  |

---

### Python Program

# Transition Probability

```
transitions = {}
tag_count = {}

for sentence in corpus:
    tags = [tag for word, tag in sentence]

    for i in range(len(tags) - 1):
        current = tags[i]
        next_tag = tags[i + 1]

        pair = (current, next_tag)
        transitions[pair] = transitions.get(pair, 0) + 1
        tag_count[current] = tag_count.get(current, 0) + 1

for (current, next_tag), count in transitions.items():
    probability = count / tag_count[current]
    print(current, "->", next_tag, probability)
```

### Q4 – Recreate the Hidden Markov Model using Python Dictionaries

HMM has two main probability components:

1. **Emission probability** → tag → word
2. **Transition probability** → tag → next tag

---

#### 1. Emission dictionary

We store probabilities like: emission

```
= {
    "DT": {"The": 0.4, "A": 0.4},
```



```
"NN": {"boy": 0.125, "cat": 0.25, "milk": 0.25},  
"VBZ": {"drinks": 0.4}  
}
```

For example:

emission["NN"]["cat"] gives:

0.25

Meaning:

---

## 2. Transition dictionary

We store probabilities between tags:

```
transition = {  
  "DT": {"NN": 1.0},  
  "NN": {"VBZ": 0.625, "NNS": 0.125},  
  "VBZ": {"NN": 0.8},  
  "NNS": {"VBP": 0.6},  
  "VBP": {"NN": 0.5, "NNS": 0.25, "RB": 0.25}  
}
```

For example:

transition["NN"]["VBZ"] gives:

0.625

Meaning:

## Complete HMM Structure

# HMM using dictionaries

emission =

```
{ "DT": {  
  "The": 0.4,  
  "A": 0.4  
},  
  
  "NN": {  
    "boy": 0.125,  
    "girl": 0.125,  
    "rice": 0.25,  
    "cat": 0.25,  
    "dog": 0.125,  
    "teacher": 0.125,  
    "English": 0.125,  
    "milk": 0.25  
  },  
}
```

```
"VBZ": {
  "eats": 0.2,
  "drinks": 0.4,
  "chases": 0.2,
  "teaches": 0.2
},

"NNS": {
  "students": 0.2,
  "Students": 0.2,
  "Birds": 0.2,
  "Children": 0.2,
  "games": 0.2
},

"VBP": {
  "study": 0.25,
  "fly": 0.25,
  "play": 0.25
}, "RB":
{
  "high": 1.0
}
}
```

transition =

```
{ "DT": {
  "NN": 1.0
},

"NN": {
  "VBZ": 0.625,
  "NNS": 0.125
},

"VBZ": {
  "NN": 0.8,
  "NNS": 0.2
},

"NNS": {
  "VBP": 0.6
},
```

```

"VBP": {
  "NN": 0.5,
  "RB": 0.25,
  "NNS": 0.25
}
}

```

## Q5 – Viterbi Algorithm

Now we use the **HMM we created in Q4** to predict the POS tags for:

The cat drinks milk

The assessment specifically asks us to implement **Viterbi without built-in HMM/NLP libraries**.

### What is Viterbi?

The **Viterbi algorithm finds the most probable sequence of hidden states**.

Here:

- **Observed words:** The → cat → drinks → milk
- **Hidden states:** POS tags • We want the most likely sequence:

? → ? → ? → ?

---

### Step 1: Start with The

From the training corpus, the first tags are:

- DT = 5 times
- NNS = 3 times So:

For The:

Therefore: So:

The → DT

is the best possibility.

---

### Step 2: cat

We know: and:

Therefore:

So:

The → DT → cat → NN

---

### Step 3: drinks We

have:

and:

Therefore:

So:

The → DT → cat → NN → drinks → VBZ

---

#### Step 4: milk We

have:

and:

Therefore:

---

#### Final Viterbi Result

|     |     |         |      |
|-----|-----|---------|------|
| The | cat | drinks  | milk |
| ↓   | ↓ ↓ | ↓ DT NN |      |
|     | VBZ | NN      |      |

#### Predicted POS sequence:

This matches the grammatical structure represented in the training corpus.

---

#### Python Program

# Viterbi Algorithm

```
words = ["The", "cat", "drinks", "milk"]
```

```
states = ["DT", "NN", "VBZ", "NNS", "VBP", "RB"]
```

```
start = {  
    "DT": 5 / 8,  
    "NNS": 3 / 8  
}
```

```
viterbi = {}  
path = {}
```

# First word for

state in states:

```
    emission_prob = emission.get(state, {}).get(words[0], 0)  
viterbi[state] = start.get(state, 0) * emission_prob    path[state]  
= [state]
```

# Remaining words

```
for i in range(1, len(words)):
```

```
    new_viterbi = {}  
    new_path = {}
```

```

for current in states:
    emission_prob = emission.get(current, {}).get(words[i], 0)

    best_probability = 0
    best_previous = None

    for previous in states:
        transition_prob = transition.get(previous, {}).get(current, 0)

        probability =
( viterbi[previous]
* transition_prob
* emission_prob
    )

        if probability > best_probability:
            best_probability = probability
            best_previous
            = previous

    new_viterbi[current] = best_probability

    if best_previous is not None:
        new_path[current] = path[best_previous] + [current]

    viterbi = new_viterbi
    path = new_path

# Find best final state best_state =
max(viterbi, key=viterbi.get)

print("Best POS sequence:") print(path[best_state])

print("Probability:", viterbi[best_state])

```

### Output

Best POS sequence:  
['DT', 'NN', 'VBZ', 'NN']

Probability: 0.003125